ReasonLink: Characterizing and Optimizing the Handoff Between Neural and Symbolic Accelerators

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
P1: Exploration & Progress | Due October 26, 2026, 11:59 PM ET

I. Project Direction and Updates

Neurosymbolic AI is an emerging compositional paradigm that fuses neural learning with symbolic reasoning to enhance the transparency, interpretability, and trustworthiness of AI. It also exhibits higher data efficiency making it promising for edge deployments [1]. Because the neural stage is compute bound, and the symbolic stage is memory bound, heteregenous hardware deployment, such as a specialized accelerator for each stage, is a natural fit. However, the interface between the two accelerators can become a bottleneck, limiting end-to-end performance.

We therefore ask: *after both ends of a neuro-symbolic pipeline are accelerated, does the interface between them become the bottleneck, and if so, which interface mechanisms remove it?*

Visual Sudoku illustrates the problem. A neural network recognizes handwritten digits, and a SAT solver uses the recognized givens to complete the grid, following the task studied in SATNet [2]. The Sudoku rules are identical across puzzles; only the givens change. Yet SAT-Accel's host interface [3] constructs and transfers a complete solver memory image for each instance. In our implementation, a puzzle with 36 recognized givens produces an image containing roughly 632 KB of data. Much of this work reconstructs and moves information that the solver already received for the preceding puzzle.

Our eventual goal to solve this issue, if it exists, is to enable *producer–consumer pipelining* between the neural and symbolic accelerators: predictions should flow into the solver's input path as they are produced, allowing inference, transfer, and solver-state preparation to overlap. ReasonLink will explore compact literal streams, on-chip buffering, and resident base constraints to support this execution model. The current milestone focuses on the infrastructure and baselines needed to evaluate that design. We have implemented serial and double-buffered software paths; double buffering overlaps conversion and transfer for puzzle $i{+}1$ with solving puzzle i. These baselines establish functional correctness and quantify the overhead that the eventual pipelined interface must reduce.

Changes since the proposal. We moved from Chipyard/Gemmini to ESP [4] with an Ariane core and NVDLA *nv_small* (int8). Immediate board access was unavailable, while the departmental server already provided the ESP toolchain and an existing NVDLA integration path. VCU118 execution remains a later target. SAT-Accel [3] remains the intended symbolic accelerator; the workload, research question, and interface mechanisms are unchanged.

We also separated software exploration from hardware validation. The proposal targeted accelerator execution for P1; currently, neural outputs are replayed from an int8 emulator, and MiniSat [5] replaces SAT-Accel after decoding its memory image. This enables functional checks and interface experiments while integration is blocked. It does *not* establish accelerator latency or system speedup. P1's contribution is a validated software path and evidence that guides the interface design.

II. Exploration and Findings

Locating the interface cost. Inspection of SAT-Accel's host flow and our image-builder experiments showed that each puzzle rebuilds the same 729-variable, 11,988-clause base formula with a small set of new givens. For a representative puzzle with 36 givens, the replayed score array is 1,296 bytes, but the solver image occupies about 632 KB of data within 10.5 MB of allocated buffers. This suggests that compressing scores alone misses the main opportunity. Our current direction is to retain the base formula and send only the changing givens. A compact stream requires corresponding solver-side loading and reset support; a FIFO alone cannot remove full-image reconstruction.

Software optimization and overlap. We compared serial and double-buffered handoffs, each in two modes. *Faithful* reproduces SAT-Accel's full-buffer clearing and copying; *lean* clears and copies only used regions. Both use busy polling. Lean provides a stronger software baseline against which hardware changes must be justified. Double buffering uses two explicitly owned slots so conversion and copying for puzzle $i{+}1$ overlap solving puzzle i.

Table I shows that used-region handling reduces serial wall time from 2.81 to 1.45 ms per puzzle, before adding any ReasonLink hardware. Argmax takes less than 1 μs; almost all conversion time is image construction. In serial runs, conversion, copying, and synchronization account for 73.6% of their combined time with solving in faithful mode and 50.5% in lean mode. These fractions exclude neural inference.

Double buffering barely improves faithful wall time but reduces lean wall time to 0.80 ms per puzzle. The exposed fraction, defined as $(T_{\text{wall}} - \sum T_{\text{solve}})/T_{\text{wall}}$, falls from 70.5% to 5.1% across those modes. Thus overlap depends on the

TABLE I
PILOT MEASUREMENTS ON X86 UNDER WSL2, WITH REPLAYED SCORES AND A MINISAT BACKEND. PHASE COLUMNS ARE MEDIANS IN μS; WALL TIME IS TOTAL RUN TIME DIVIDED BY 1,000 PUZZLES. EACH CONFIGURATION HAS ONE MEASURED RUN AFTER 20 WARM-UP PUZZLES.

Configuration	T_{conv}	T_{copy}	T_{solve}	**Wall**
Serial, faithful	1112	880	728	2809
Serial, lean	696	13	726	1450
Double-buffered, faithful	1461	1159	764	2703
Double-buffered, lean	711	16	743	801

relative stage costs, while lean still copies roughly 632 KB per puzzle. These observations motivate testing base-formula reuse *in software first*, then measuring the additional benefit of a direct hardware stream. They are pilot evidence for the proposal's handoff and overlap hypotheses, not confirmation on ESP.

Workload limits. The retrained classifier achieves 99.26% given-cell accuracy, but only 79.0% of puzzles recover the ground-truth grid; 19.2% become UNSAT and 1.8% are SAT with a wrong grid. Small perception errors compound across givens. We will therefore hold predictions fixed across interfaces and distinguish task accuracy from interface correctness. Separately, the current 24–40-clue puzzles require only 1.17 mean MiniSat decisions. They exercise loading more than search, so a harder workload is needed to test whether ReasonLink's benefit shrinks with longer solves.

III. IMPLEMENTATION AND INFRASTRUCTURE PROGRESS

Working software and tools. We implemented the Sudoku encoder, 1,000 unique-solution puzzles with MNIST test images, classifier training and int8 emulation, and a PySAT [6] reference path. ESP's demo weights achieved only 74.5% MNIST accuracy under the evaluated preprocessing; investigation found a mismatch between training preprocessing and the deployed int8 input path. Retraining the same architecture raised emulated int8 accuracy to 99.20%. The emulator matches ESP's published reference output; the retrained model's hardware output remains unverified.

The C image builder matches SAT-Accel's original host implementation byte for byte in the compared initialized regions on all 13 test cases. The image-decode–MiniSat path agrees with the reference SAT/UNSAT result on all 1,000 puzzles. It returns 992 identical grids and eight alternative grids verified to satisfy their instances. All four baseline configurations return identical results to each other. These checks validate the tested software path; they do not exercise SAT-Accel's kernels.

The baseline harness records conversion, copying, synchronization, and solving with cycle counters. Static RISC-V builds are available; a local QEMU check matched native results on 50 puzzles. ESP is pinned to release 2026.1.0, and the server has Vivado/ModelSim 2023.2 and RISC-V GCC 9.2. Recompiling ESP's demo reproduced its NVDLA loadable byte for byte, and our retrained classifier also compiles for *nv_small*.

Under development and technical barriers. Neither accelerator has run in our ESP pipeline, and ReasonLink RTL is not implemented. The server lacks `v++`/XRT for SAT-Accel's emulation flow; its free-running HLS processes also prevent straightforward sequential C simulation. A threaded kernel harness or access to the required emulation stack is needed. ESP execution must additionally confirm the real NVDLA output layout and RISC-V ISA/ABI compatibility. Current copy timings are CPU memory copies, not DMA/NoC measurements; solving includes image decoding and MiniSat loading. Single-run x86 timings cannot predict ESP performance.

Code, raw results, and reproduction details are maintained at https://github.com/AaronCherian404/ReasonLink in `workload/`, `handoff/`, `baselines/`, `results/p1/`, and `PROGRESS.md`.

IV. NEXT STEPS

For the November 6 midterm: present the revised platform, validated software pipeline, and pilot comparison, with a clear account of the remaining hardware gap. Repeat the baseline measurements to assess variability and implement a CPU-only resident-base/delta-update prototype. Check it against full-image rebuilding on the same predictions, reporting correctness, bytes moved, and conversion time. This will establish how much benefit is available from software reuse before attributing gains to hardware.

For Milestone 2 on November 9: prioritize functional SAT-Accel execution through a threaded HLS harness or a compatible emulation environment, and verify the classifier output on ESP if simulation is ready. Target a compact-literal interface prototype with FIFO backpressure, instance boundaries, and explicit reset behavior. Validate that consecutive puzzles preserve constraints and clear prior assignments and instance-dependent learned clauses before measuring speed. If SoC integration remains blocked, report module-level functional results and outstanding dependencies separately from host timing.

After Milestone 2, integrate resident-state updates and overlap, compare against both lean baselines, and introduce harder instances to study the solve-time crossover. The final evaluation will include neural execution, DMA/NoC traffic, and interface stalls once measurable. Reproducible code and experiment scripts will accompany the final report on December 18.

REFERENCES

[1] Z. Wan, H. Yang, R. Raj, C.-K. Liu, A. Samajdar, A. Raychowdhury, and T. Krishna, "CogSys: Efficient and scalable neurosymbolic cognition system via algorithm-hardware co-design," in *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025. [Online]. Available: https://zishenwan.github.io/publication/HPCA25_CogSys.pdf

[2] P.-W. Wang, P. L. Donti, B. Wilder, and J. Z. Kolter, "SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[3] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[4] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with open ESP," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, 2020. [Online]. Available: https://www.esp.cs.columbia.edu

[5] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT)*, ser. LNCS, vol. 2919, 2003, pp. 502–518.

[6] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python toolkit for prototyping with SAT oracles," in *Theory and Applications of Satisfiability Testing (SAT)*, 2018, pp. 428–437.